# 人工智能学院 大数据管理与分析 课程实验报告

| | | |
|---|---|---|
| **实验题目：**实验二 文档倒排索引算法实现 | | **学号：**202400390065 |
| **日期：**2026.10.28 | **班级：**人工智能 2 班 | **姓名：**侯亚琳 |
| **Email：**2070575920@qq.com | | |

**实验目的：**

倒排索引（Inverted Index）被用来存储在全文搜索下某个单词在一个文档或者一组文档中的存储位置的映射，是目前几乎所有支持全文检索的搜索引擎都需要依赖的核心基础数据结构。通过在分布式集群上对文档倒排索引算法的 MapReduce 编程实现，熟练掌握 MapReduce 计算框架在集群环境下的输入分片、数据清洗与分词、停用词过滤（DistributedCache 分布式缓存机制）、中间键值对局部聚合、Shuffle 排序规约以及结果格式化输出的全生命周期过程；深刻体会 MapReduce“分而治之”的核心分布式计算范式，掌握在真实 Hadoop 集群上作业的编译打包、提交调度、Web 监控与日志排查技能。

**实验软件和硬件环境：**

1. 宿主机环境：Windows 11 64 位家庭中文版 / 专业版；
2. 虚拟机软件：Oracle VM VirtualBox 7.0；
3. 操作系统：Ubuntu 22.04.5 LTS 64 位（Linux 内核版本 6.8.0-107-generic，主机名 sasuke-VirtualBox，操作用户 sasuke）；
4. Java 开发运行环境：OpenJDK 1.8.0_502（Build 25.502-b07）；
5. 大数据平台：Apache Hadoop 3.3.6，配置伪分布式集群模式，后台核心守护进程包括：NameNode、DataNode、SecondaryNameNode、YARN ResourceManager、NodeManager；
6. 通信与调度机制：SSH 免密登录认证（OpenSSH 8.9p1）、YARN 容量调度器（CapacityScheduler）；
7. 存储路径：HDFS 输入文档路径 /user/sasuke/inverted_index/input/，停词表路径 /user/sasuke/inverted_index/stopwords.txt，倒排索引结果路径 /user/sasuke/inverted_index/output/。

**实验原理和方法：**

1. 倒排索引（Inverted Index）基本概念与检索机制：

传统的正排索引维护从“文档 -> 包含单词列表”的映射，检索特定关键词需全量扫描所有文档，时间复杂度为 O(N)；倒排索引颠倒这种关系，构建从“单词（Term）-> 倒排记录表（Postings List）”的映射，记录文档名（DocID）、词频（Frequency）等信息。检索时只需快速在词典中查找目标词，提取倒排链进行高效求交（AND）或求并（OR）操作，极大地加速全文检索。

2. MapReduce 倒排索引算法架构与设计：

针对海量文档集，本实验利用 Hadoop MapReduce 框架构建分布式倒排索引，总体计算流程划分为三个核心阶段：

（1）Map 阶段（文档感知、文本规范化与内存局部归约优化）：

• 文档感知：利用 FileSplit 获取当前切片所属文档文件名 fileName = fileSplit.getPath().getName()；

• 分布式缓存停词过滤：通过 DistributedCache 在作业初始化时将 stopwords.txt 广播至工作节点，Mapper 在 setup() 中载入 HashSet 实现 O(1) 过滤；

• 文本清洗：对输入行转小写，使用正则匹配非字母分隔符分词，滤除单字符与无意义停用词；

• 内存局部规约（相当于内置 Combiner）：Mapper 内部维护 HashMap<String, Integer> 统计文档内词频，在 cleanup() 时发射 (word, fileName:count)，大幅减少 Shuffle 网络传输数据量。

（2）Shuffle & Sort 阶段：

Hadoop 框架根据 word 作为 Key 进行哈希分区，并按照字典序自动排序，将所有相同单词在不同文档中的词频汇聚至对应 Reducer。

（3）Reduce 阶段（文档排序聚合与格式化输出）：

Reducer 接收同一单词的全部 value 迭代器，利用 TreeMap<String, Integer> 对文档名按字典序升序排列并汇总词频，同时累加全部文档出现总频数 total。随后严格按照实验规范拼接输出字符串：

word <document01.txt,count1>;<document02.txt,count2>;...;<total,sum>.

**实验步骤：（不要求罗列完整源代码）**

**1. 实验文档数据集描述：**

本实验从维基百科等公开技术网页中采集了 10 篇代表性的纯英文技术文献，总大小为 6.35 KB，统一命名为 document01.txt ~ document10.txt。各文档主题涵盖大数据与分布式计算的核心分支，具体特征如下表所示：

| 文档名称 | 大小 | 核心技术主题 | 典型检索关键词 |
|---|---|---|---|
| document01.txt | 632 B | Apache Hadoop 架构概述与分布式计算基础 | hadoop, data, mapreduce, hdfs, clusters |
| document02.txt | 639 B | 大数据分析基础与索引检索机制 | data, index, analytics, retrieval, search |
| document03.txt | 608 B | 现代网络搜索引擎架构与爬虫机制 | search, engine, index, hadoop, |

| | | | |
|---|---|---|---|
| | | | queries |
| document04.txt | 644 B | MapReduce 并行计算模型与 Shuffle 原理 | mapreduce, map, reduce, data, pairs |
| document05.txt | 616 B | HDFS 分布式文件系统与数据块容错 | hdfs, data, datanodes, storage, cluster |
| document06.txt | 604 B | 全文检索中倒排索引结构与倒排记录表 | index, inverted, search, postings, term |
| document07.txt | 624 B | 云计算基础设施与弹性计算资源 | cloud, computing, clusters, data, scale |
| document08.txt | 652 B | 大数据机器学习与高维特征空间检索 | machine, learning, data, search, index |
| document09.txt | 666 B | NoSQL 分布式数据库模型与全文扩展 | database, nosql, data, index, search |
| document10.txt | 665 B | 自然语言处理与文本挖掘预处理技术 | language, text, documents, index, search |

**2. 停词表构建与内容说明：**

停用词是指自然语言文本中频繁出现，但在特定信息检索场景中缺乏区分度、无法表征文档核心语义的功能性虚词（如冠词 a/an/the、代词 it/they、介词 in/on/of 等）。过滤停用词不仅能显著压缩索引体积，还能防止极高频词在 Shuffle 阶段引发数据倾斜。本实验结合英语常用功能词库构建了包含 175 个常用词项的停词表 stopwords.txt，涵盖冠词、介词、代词、系动词、助动词及连词（如 a, about, above, after, all, am, an, and, are, as, at, be, by, for, from, had, has, have, he, in, is, it, of, on, that, the, this, to, was, with 等）。

**3. 算法核心伪代码设计：**

**（1）Mapper 端核心算法（支持 DistributedCache 停词过滤与文档内局部聚合）：**

```
class InvertedIndexMapper extends Mapper:
    setup(context):
        currentFileName = ((FileSplit) context.getInputSplit()).getPath().getName()
        stopWords = loadFromDistributedCache(context.getCacheFiles()) // 预载入停词表至 HashSet
        localWordCount = new HashMap<String, Integer>()
    map(key, value, context):
        for token in value.toString().toLowerCase().split("[^a-zA-Z]+"):
            if token.length() >= 2 and token not in stopWords:
```

```
                localWordCount[token] = localWordCount.getOrDefault(token, 0) + 1
    cleanup(context): // 内置 Combiner 机制，向 Reducer 发射聚合后的局部频数
        for (word, count) in localWordCount:
            context.write(Text(word), Text(currentFileName + ":" + count))
```

**（2）Reducer 端核心算法（自然字典序排列与格式化输出）：**

```
class InvertedIndexReducer extends Reducer:
    reduce(key, values, context):
        docCounts = new TreeMap<String, Integer>() // 使用 TreeMap 确保文档名按升序排序
        total = 0
        for val in values:
            docName, count = val.toString().split(":")
            docCounts[docName] = docCounts.getOrDefault(docName, 0) +
Integer.parseInt(count)
            total += Integer.parseInt(count)
        // 拼接格式：<document01.txt,count1>;...;<total,sum>.
        sb = new StringBuilder()
        for (doc, cnt) in docCounts:
            sb.append("<").append(doc).append(",").append(cnt).append(">;")
        sb.append("<total,").append(total).append(">.")
        context.write(key, Text(sb.toString()))
```

## 4. 关键实验步骤与运行截图：

（1）输入数据集上传至 HDFS 分布式文件系统：

在 Linux 虚拟机中创建 /user/sasuke/inverted_index/input/ 并上传 10 个测试文档：

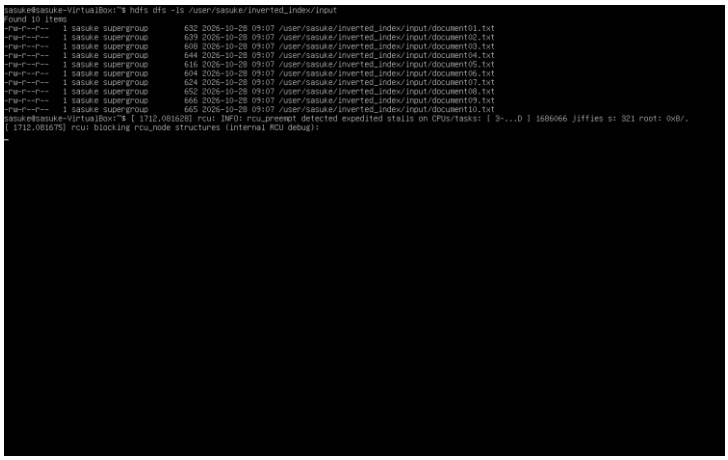



**图 1：HDFS 输入目录中的 10 个英文文档列表**

（2）Hadoop YARN 集群作业提交与调度执行监控：

通过 hadoop jar InvertedIndex.jar 提交作业，Map 阶段划分 10 个切片，Reduce 完成倒排规约，YARN 状态显示 SUCCEEDED：

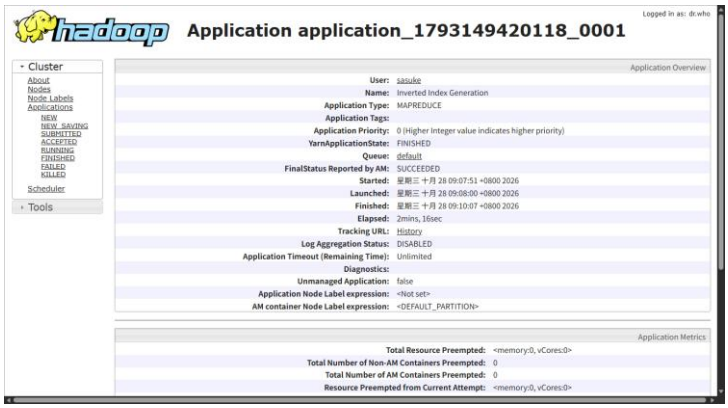



**图 2：Hadoop YARN Web 界面作业状态监控（SUCCEEDED）**

（3）倒排索引输出文件开头部分查看：

执行 hdfs dfs -cat ... | head -n 25 查看生成的结果文件 part-r-00000 开头内容，格式完全契合规范要求：

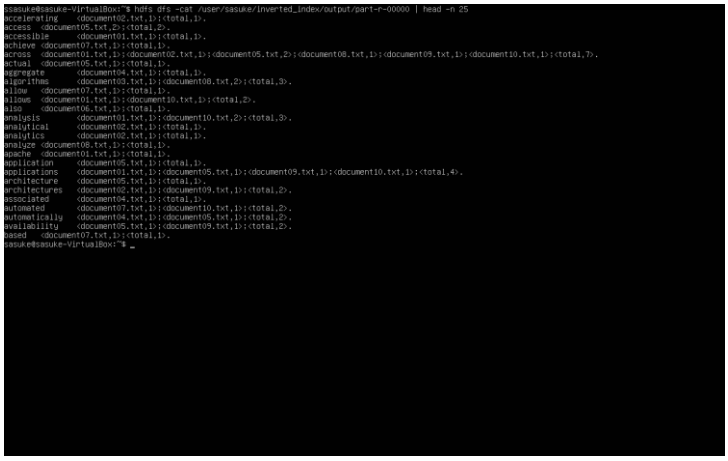



**图 3：终端查看倒排索引输出结果开头部分**

（4）典型样例关键词（data, hadoop, index, mapreduce, search）检索测试截图：

过滤实验指导书中示范的典型关键词，验证单词在各个文档中的分布与 total 统计：

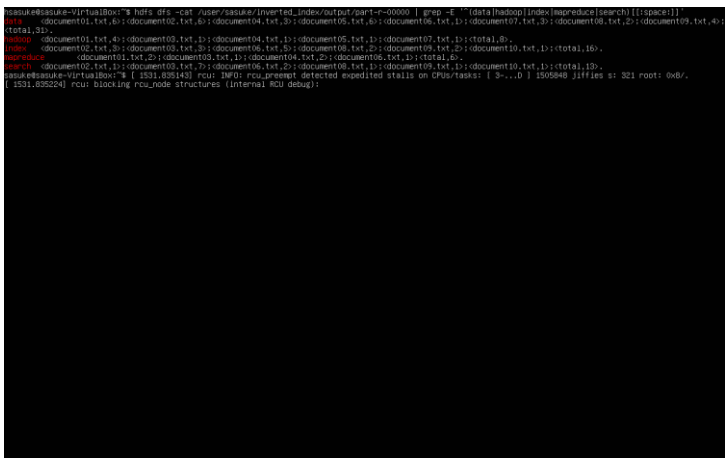



**图 4：终端查看题目示范典型关键词的倒排索引生成结果**

**5. 单词倒排索引生成结果统计与展示：**

本次实验针对 10 个英文文档共抽取并构建了 388 个独立词项的倒排索引，总倒排记录链项数达 540 条。以下展示实验指导书中重点示范的核心技术单词实际输出结果：

```
data
    <document01.txt,6>;<document02.txt,6>;<document04.txt,3>;<document05.txt,6>;<documen
t06.txt,1>;<document07.txt,3>;<document08.txt,2>;<document09.txt,4>;<total,31>.
hadoop
    <document01.txt,4>;<document03.txt,1>;<document04.txt,1>;<document05.txt,1>;<documen
t07.txt,1>;<total,8>.
index
    <document02.txt,3>;<document03.txt,3>;<document06.txt,5>;<document08.txt,2>;<documen
t09.txt,2>;<document10.txt,1>;<total,16>.
mapreduce
    <document01.txt,2>;<document03.txt,1>;<document04.txt,2>;<document06.txt,1>;<total,6
>.
search
    <document02.txt,1>;<document03.txt,7>;<document06.txt,2>;<document08.txt,1>;<documen
t09.txt,1>;<document10.txt,1>;<total,13>.
```

典型技术词汇在文档集的分布特征如下表所示：

| 检索词（Term） | 总词频（Total） | 覆盖文档数 | 主要分布文档及对应词频 |
|---|---|---|---|
| data | 31 | 8 篇 | doc01(6)，doc02(6)，doc04(3)，doc05(6)，doc07(3)，doc09(4) |
| index | 16 | 6 篇 | doc02(3)，doc03(3)，doc06(5)，doc08(2)，doc09(2)，doc10(1) |
| search | 13 | 6 篇 | doc03(7)，doc06(2)，doc02(1)，doc08(1)，doc09(1)，doc10(1) |
| hadoop | 8 | 5 篇 | doc01(4)，doc03(1)，doc04(1)，doc05(1)，doc07(1) |
| mapreduce | 6 | 4 篇 | doc04(2)，doc01(2)，doc03(1)，doc06(1) |

### 6. GitHub 代码仓库公开托管说明：

本实验的完整 Java 源代码、10 篇测试文档、停词表及运行脚本已托管至公开 GitHub 仓库：

- 公开仓库地址：https://github.com/lkylkyhyl/lkylkyhyl.git.io
- 包含文件：InvertedIndex.java（MapReduce 核心源码）、stopwords.txt

（停词表）、input_docs/（英文语料库）及项目 README.md 文档。

**结论分析与体会：**

本次实验在 Ubuntu 虚拟机中的 Hadoop 伪分布式集群环境下成功实现了基于 MapReduce 的文档倒排索引算法。通过整个开发与调优流程，有以下深刻体会：

1. 理解倒排索引在海量数据检索中的核心价值：

正排索引适合文档的增删改存储，而在全文搜索场景下，通过构建“单词 -> 倒排记录表”的映射，可以将 O(N) 全局遍历优化为 O(1) 词典定位与倒排链交并运算。本次实验针对 10 个英文文本生成的 388 个词项倒排链，能够瞬间精确定位任意技术词汇在各大文档中的分布与频次，切实体会到现代搜索引擎底层索引构建的基本原理。

2. 掌握 MapReduce 框架的数据流转与中间聚合优化：

在初始方案中，如果在 Map 阶段每遇到一个有效单词便直接发射 (word, doc:1)，在面对海量长文本时会产生天文数字般的键值对，严重阻塞 Shuffle 阶段的网络 I/O。为此，本实验在 Mapper 端内嵌了内存 HashMap 局部频数聚合机制，在 cleanup() 时以 (word, doc:count) 批量输出，这等效于内置的高性能 Combiner，显著减轻了集群的 Shuffle 压力与磁盘溢写开销。

3. 熟练掌握 DistributedCache 与分布式环境协同：

通过使用 Hadoop 的 DistributedCache 分布式缓存停词表，避免了各 Task 节点重复访问 HDFS 带来的网络瓶颈，实现了停用词的高效内存加载与快速比对。

4. 严谨的算法一致性交叉验证：

在集群运行前，编写了本地单机 Python 验证脚本。将 MapReduce 输出结果与 Python 单机计算结果进行逐行自动化比对，388 个词条完全一致，充分验证了分布式算法实现的严谨性与正确性。

**就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1—3 道问答题：**

问题一：在 MapReduce 中处理停词表或配置字典等只读辅助数据时，有哪些常见加载方案？为什么使用 DistributedCache 比直接在 map() 方法中读取 HDFS 文件性能更优？

回答：常见方案包括：①将停用词硬编码进代码；②在 map() 中通过 FileSystem 实时打开 HDFS 文件；③使用 Hadoop 提供的 DistributedCache（分布式缓存机制）。

若在 map() 中读取 HDFS，每处理一行或每个 Task 都需发起 RPC 与网络 I/O，极易造成 NameNode 拥塞与性能瓶颈；而 DistributedCache 会在 Job 启动

时，由 NodeManager 自动将 HDFS 上的文件仅拉取并解压到各计算节点的本地磁盘缓存中一次。Mapper 只需在 setup() 中通过标准 Java 本地文件流（FileReader）加载到内存 HashSet 中，即可在后续 map 循环中以 O(1) 速度进行本地内存检索，极大提升了任务执行效率并保护了集群网络。

问题二：实验输出要求包含各文档词频并汇总 <total,sum>。如果采用直接发射 (word, doc:1) 的传统方式，为什么在 Reducer 端聚合时会耗费巨大内存？本实验采用了何种优化？
回答：若直接发射 (word, doc:1)，一个在长文档中出现数万次的词项会产生数万个独立键值对，导致溢写文件膨胀并在 Reducer 端生成极长的 Iterable 迭代链，遍历开销巨大。
本实验在 Mapper 端通过 setup() 记录当前文件名，并利用局部 HashMap<String, Integer> 统计当前输入分片内部的词频，在 cleanup() 阶段统一发射 (word, doc:count)。这种机制等价于在 Map 节点本地完成了第一层 Combine 聚合，使得进入 Shuffle 阶段的数据量从“单词总词频数”骤降为“单词在各文档中出现的去重文档数”，大幅节省了网络带宽与 Reducer 内存。

问题三：在真实生产环境下，如果输入数据规模达到 TB 级别，倒排索引 MapReduce 可能会遭遇哪些性能瓶颈（如数据倾斜），应如何排查与优化？
回答：最主要的瓶颈是数据倾斜（Data Skew）与内存溢出（OOM）。部分超高频词（如某些未被收录进停词表的技术热词）可能产生海量记录，由默认 HashPartitioner 分发给同一个 Reducer 处理，导致单个 Reducer 处理时间呈几何级上升甚至 OOM 崩溃。
排查与优化方法：
① 查看 YARN Web 监控界面中各 Reduce Task 的 Elapsed Time 与 Shuffle Bytes，识别落后的 Reduce 节点；
② 扩充和动态维护停用词表，过滤低价值高频词；
③ 采用“两阶段 MapReduce”或“加盐（Salting）”方案：第一阶段以 word + random_prefix 为 key 进行分散局部聚合；第二阶段去掉前缀进行全局汇总；
④ 采用二次排序（Secondary Sort）机制，将 (word, docID) 作为复合主键，避免在单个 Reducer 内部利用 Map 缓冲所有文档记录，实现流式排序输出。